

Parte A

1. Insertion sort es el que hace menos trabajo en un conjunto de datos ya ordenados debido a que es el que hace menos comparaciones respecto a bubble sort y selection sort
2. Conjunto de datos en el que quicksort se comporta pésimo con el primer elemento como pivote
un arreglo que ya esté ordenado, se usa ejemplo de retos
`arrayOrdenado = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`
3. Ordenar y luego Binario debido a que se tardaría 100 veces logaritmo natural de los datos mas el ordenamiento
mientras que una búsqueda secuencial se tardaría 100 veces el número de datos

Parte B

Se usa un conjunto de 100 registros o datos

Dato	Pasos Secuencial	Tiempo Secuencial	Pasos Binaria	Tiempo binaria
Primero	1	$2 \cdot 10^{-6}$	6	$3.90 \cdot 10^{-6}$
Medio	50	$2.50 \cdot 10^{-6}$	1	$1.2 \cdot 10^{-6}$
Último	100	$3.69 \cdot 10^{-6}$	7	$2.4 \cdot 10^{-6}$
No existe	100	$4 \cdot 10^{-6}$	7	$1.4 \cdot 10^{-6}$

Parte C

Comparación entre 10 registros o datos ordenados o no ordenados

Ordenados

Array	Insertion	Bubble	selection
Comparaciones	9	9	45
Intercambios	0	0	0

Desordenados

Array	Insertion	Bubble	selection
comparaciones	42	45	30
Intercambios	24	8	24

Parte D

Datos	Bubble	Bucket
20	3.8623809814453125e-05	4.1484832763671875e-05
40	9.179115295410156e-05	5.054473876953125e-05
60	0.0001461505889892578	4.315376281738281e-05

Los tiempos crecen distintos debido a que estos tienen una diferente complejidad donde Bubble presenta un $O(n^2)$, mientras que bucket presenta una complejidad de $O(n+k)$ lo cual quiere decir que a medida que aumenten los datos a valores mucho mayores bubble va a crecer mucho más rápido que bucket

Parte E

Complejidad en C++ = 30

Complejidad en Python = 35

La diferencia de complejidad entre los dos lenguajes que fue dada por Codacy se puede deber principalmente a que en python no existen tantas funciones y un proceso que se podía reducir como el de registrar datos del ordenamiento $O(n)$ se hace múltiples veces en python, esto se confirmó parcialmente con SonarQube el cual dio una complejidad de 30 y 33 respectivamente.

Parte F

No estoy de acuerdo con la afirmación ya que el CI/CD no es solo construir en cambio es testear que este bien y luego hacer el merge en la rama principal en el paso del CI, para luego subir esto al entorno y que lo pueda usar el publico

Pipeline C++

Build-> Test -> Merge -> Repo -> Deployment (entorno se puede usar por los usuarios)

Pipeline Python

Test -> Merge -> Repo -> Deployment (entorno se puede usar por los usuarios)

Se puede implementar CD/CI en el laboratorio subiendo por ejercicios